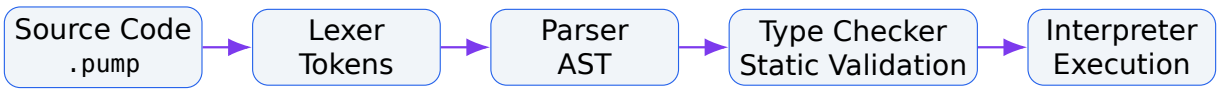
PumpScript

A Domain-Specific Language for Workout Routines

Technical Design and Implementation Report

Author	Yusuf Eren Nalbant
Course	CSE 341 – Concepts of Programming Languages
University	Gebze Technical University
Implementation Language	Python 3.8+
Project Type	Domain-Specific Language (DSL)



June 2026

Contents

1 Project Overview

Technical Summary

PumpScript is a small domain-specific language designed to express workout routines in a safe, readable, and semantically constrained form. The implementation is written in Python without external parsing libraries. It consists of a hand-written lexer, a recursive-descent parser, a class-based Abstract Syntax Tree, a two-pass static type checker, and a tree-walking interpreter.

The goal of PumpScript is not to reproduce every feature of a general-purpose language. Instead, the language maps concepts from the workout domain directly into language constructs:

- `routine`: defines a parameterized workout procedure.
- `exercise`: creates a workout record with a fixed field schema.
- `rest N seconds`: represents a domain-specific rest instruction.
- `if/else`: selects a block using a Boolean condition.
- `repeat N times`: executes a block a fixed number of times.

The language supports the primitive types **int**, **float**, **string**, and **bool**. Type mismatches are rejected before execution. The only implicit conversion is the safe widening conversion from **int** to **float**.

1.1 Implemented Features and Deliberate Non-Goals

Implemented		Outside the Current Scope	
Lexical analysis	Tokens, comments, and source-line tracking	**Arrays**	No array type or indexing
Recursive-descent parsing	EBNF-aligned AST construction	**Assignment**	No mutable assignment expression
Static typing	Fields, parameters, calls, and expressions	**Return values**	Routines are procedures
Static scoping	Global routines and local parameter environments	**Field access**	Exercise fields cannot be read later
Tree-walking execution	Direct AST interpretation	**Boolean connectives**	No `and`, `or`, or `not`

These limits keep the language small, testable, explainable, and aligned with the core programming-language concepts targeted by the project.

2 System Architecture

PumpScript combines a compilation-like front end with an interpretation phase. The source program must pass lexical, syntactic, and static-semantic validation before the interpreter is invoked.

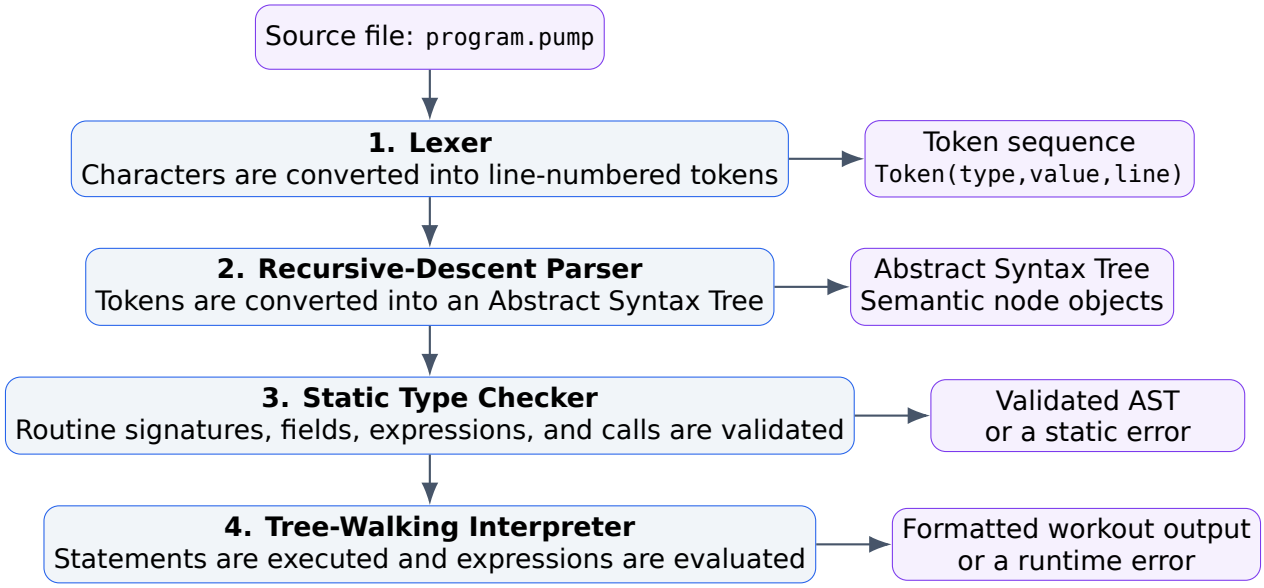


Figure 1: End-to-end PumpScript processing pipeline. The vertical layout prevents node and arrow overlap.

2.1 Source File Organization

File	Responsibility
lexer.py	Converts source text into tokens and reports illegal characters or unterminated strings.
parser.py	Builds the AST from the token stream using recursive descent.
ast_nodes.py	Defines program, declaration, statement, and expression node classes.
type_checker.py	Registers routine signatures and validates fields, calls, and expression types.
interpreter.py	Executes AST nodes, manages environments, calls routines, and enforces runtime domain constraints.
pumpscript.py	Provides file loading, pipeline sequencing, exception handling, and the `--dump-ast` command-line option.
examples/	Contains valid programs and negative lexer, parser, and type-checking examples.

2.2 Two-Pass Processing

Both the type checker and the interpreter register routine definitions before processing executable statements:

1. **Pass 1:** collect every routine name and parameter signature in a global table.
2. **Pass 2:** check or execute routine bodies and top-level statements.

This design supports forward calls. A routine may call another routine that appears later in the source file because all routine signatures and declarations are available before body traversal begins.

3 Lexical Analysis

The lexer reads the source from left to right and converts each meaningful fragment into a `Token(type, value, line)` object. Line information is preserved so that every later phase can report source-oriented errors.

3.1 Token Categories

Category	Form or Values	Example
Keyword	`routine`, `exercise`, `rest`, `repeat`, `times`, `if`, `else`, `seconds`, type names, and Boolean constants	`routine`, `false`
Identifier	`[A-Za-z_][A-Za-z0-9_]*`, excluding keywords	`bench_press`, `sets`
Integer literal	One or more decimal digits	`4`, `90`
Float literal	Digits, a decimal point, and at least one fractional digit	`80.0`, `2.5`
String literal	Text enclosed in double quotes without crossing a line boundary	`"chest"`
Operator	`+ - * / > < >= <= == !=`	`sets + 1`
Separator	`{ } ( ) : ,`	Parameter and block syntax
Comment	From `//` to the end of the current line	`// Chest day`
EOF	Artificial token marking the end of input	`Token(EOF, "", line)`

3.2 Maximal Munch and Match Priority

The lexer emits the longest valid token at the current position. For example, `3.14` is recognized as one `FLOAT_LIT`, not as two integers separated by a dot. Two-character operators are tested before one-character alternatives, preventing `>=` from being split incorrectly.

The language and token set are intentionally small. A hand-written scanner avoids generator dependencies and makes character advancement, lookahead, maximal munch, keyword recognition, and line tracking explicit in the implementation.

3.3 Tokenization Example

```
rest 90 seconds
```

```
Token(KEYWORD, 'rest',    line=1)
Token(INT_LIT, '90',      line=1)
Token(KEYWORD, 'seconds', line=1)
Token(EOF,     '',        line=1)
```

4 Syntax and the Recursive-Descent Parser

The parser consumes the token sequence and determines whether it conforms to the language grammar. A successful parse returns an AST. A failure raises a `ParseError` that identifies the source line, expected token, and encountered token.

4.1 Complete EBNF Grammar

```
<program>        = { <routine_decl> | <statement> } EOF

<routine_decl>   = "routine" IDENT "(" [ <param_list> ] ")" <block>
<param_list>     = <param> { "," <param> }
<param>          = IDENT ":" <type>
<type>           = "int" | "float" | "string" | "bool"

<block>          = "{" { <statement> } "}"
<statement>      = <exercise_block>
                 | <rest_stmt>
                 | <repeat_stmt>
                 | <if_stmt>
                 | <routine_call>

<exercise_block> = "exercise" IDENT "{" { <field> } "}"
<field>          = IDENT ":" <expr>
<rest_stmt>      = "rest" INT_LIT "seconds"
<repeat_stmt>    = "repeat" INT_LIT "times" <block>
<if_stmt>        = "if" <expr> <block> [ "else" <block> ]
<routine_call>   = IDENT "(" [ <arg_list> ] ")"
<arg_list>       = <expr> { "," <expr> }

<expr>           = <comparison> { ("==" | "!=") <comparison> }
<comparison>     = <term> { (">" | "<" | ">=" | "<=") <term> }
<term>           = <factor> { ("+" | "-") <factor> }
<factor>         = <unary> { ("*" | "/") <unary> }
<unary>          = "-" <primary> | <primary>
```

```
<primary>        = INT_LIT | FLOAT_LIT | STRING_LIT
                 | "true" | "false" | IDENT
                 | "(" <expr> ")"
```

4.2 Parser Methods and Nonterminal Mapping

Each important grammar nonterminal has a corresponding parser method. For example, `parse_routine_decl()`, `parse_exercise_block()`, and `parse_if_stmt()` directly implement their respective EBNF productions.

The parser requires at most one token of lookahead for statement selection:

- If a statement begins with a keyword, the keyword selects the statement form.
- If a statement begins with an identifier, it is parsed as a routine call.
- Expression parsing is split into precedence levels, so precedence is encoded structurally rather than repaired after parsing.

4.3 Operator Precedence and Associativity

Level	Operators	Associativity	Parser Method
1 (lowest)	`== !=`	Left	`parse_expr`
2	`> < >= <=`	Left	`parse_comparison`
3	`+ -`	Left	`parse_term`
4	`* /`	Left	`parse_factor`
5 (highest)	Unary `-`	Prefix	`parse_unary`

Table 2: Expression operator precedence in PumpScript.

For example, `2 + 3 * 4 > 10` is parsed as:

$$(2 + (3 * 4)) > 10$$

5 Abstract Syntax Tree Design

The AST removes punctuation that is no longer semantically relevant and preserves only structures required by later phases. Braces and commas are not retained as nodes. Instead, blocks, parameter lists, argument lists, and exercise fields are represented directly as object relationships.

5.1 Node Hierarchy

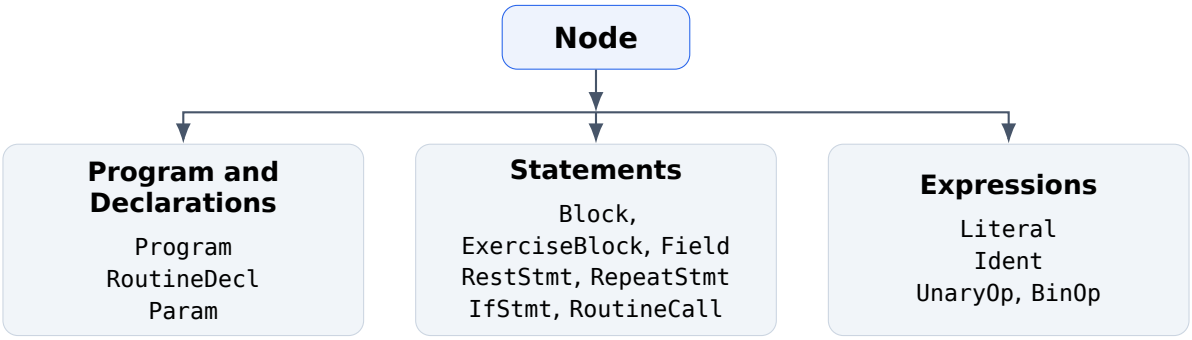


Figure 2: PumpScript AST node families. A shared branch point keeps the connectors outside the boxes.

5.2 Principal AST Nodes

Node	Information Stored
Program	Routine declarations and top-level statements.
RoutineDecl	Routine name, typed parameters, body block, and source line.
ExerciseBlock	Exercise name and its field nodes.
Field	Field name and the expression that computes its value.
RestStmt	Integer duration in seconds.
RepeatStmt	Fixed repetition count and body block.
IfStmt	Condition expression, then block, and optional else block.
RoutineCall	Routine name and argument expressions.
BinOp / UnaryOp	Operator and operand nodes.
Literal / Ident	Constant value or name reference.

Every node contains a `line` attribute so the type checker and interpreter can report the correct source line. Node `dump()` methods support the `--dump-ast` mode for readable inspection of the tree.

6 Static Type System

PumpScript uses strong static typing. Expression types are inferred before execution, and exercise fields and routine arguments are checked against declarations. Python's dynamic truthiness is not exposed to the language. For example, `if 1 {...}` is invalid because an `if` condition must have type **bool**.

6.1 Primitive Types

Type	Literal Example	Typical Use
int	`4`, `90`	Sets, repetitions, parameters, and integer arithmetic
float	`80.0`, `7.5`	Weight and mixed numeric expressions
string	`"chest"`	Text fields such as a muscle group
bool	`true`, `false`	Completion state and conditional expressions

Table 3: Primitive types supported by PumpScript.

6.2 Exercise Record Schema

Every `exercise` block must match the following named schema exactly:

Field	Expected Type	Additional Runtime Rule
`sets`	**int**	> 0
`reps`	**int**	> 0
`weight`	**float**	≥ 0
`group`	**string**	none
`completed`	**bool**	none

The type checker rejects:

- unknown field names,
- duplicate occurrences of a field,
- missing required fields, and
- field expressions whose types are incompatible with the schema.

This prevents an arbitrary key-value block from being accepted accidentally as an exercise record.

6.3 Compatibility and Coercion

Let E be the expected type and A the actual type. Compatibility is defined as:

$$\text{compatible}(E, A) = \begin{cases} \text{true}, & E = A \\ \text{true}, & E = \textbf{float} \wedge A = \textbf{int} \\ \text{false}, & \text{otherwise} \end{cases}$$

Therefore:

- **int** $\rightarrow$ **float** is allowed: `weight: 80` is valid.

- **float** $\rightarrow$ **int** is rejected: `sets: 4.0` is a type error.
- No implicit conversion exists between strings, Booleans, and numeric types.

> The widening conversion preserves numeric information, while the reverse conversion may discard a fractional component. Restricting implicit conversion to this one direction balances convenience and reliability.

6.4 Expression Typing Rules

The judgment $\Gamma \vdash e : t$ means that expression e has type t under type environment Γ.

6.4.1 Literals and Identifiers

$$\Gamma \vdash 4 : \textbf{int} \qquad \Gamma \vdash 80.0 : \textbf{float} \qquad \Gamma \vdash \texttt{"chest"} : \textbf{string} \qquad \Gamma \vdash \texttt{false} : \textbf{bool}$$

$$\frac{x : t \in \Gamma}{\Gamma \vdash x : t}$$

An identifier that is absent from the current type environment produces a static error.

6.4.2 Unary Minus

$$\frac{\Gamma \vdash e : t \qquad t \in \{\textbf{int}, \textbf{float}\}}{\Gamma \vdash -e : t}$$

Unary minus is defined only for numeric operands.

6.4.3 Arithmetic Operators

For `+`, `-`, and `*`, both operands must be numeric. The result is **float** if at least one operand is **float**; otherwise it is **int**.

$$\frac{\Gamma \vdash e_1 : t_1 \quad \Gamma \vdash e_2 : t_2 \quad t_1, t_2 \in \{\textbf{int}, \textbf{float}\}}{\Gamma \vdash e_1 + e_2 : \text{promote}(t_1, t_2)}$$

Division always produces **float** after both operands have been verified as numeric:

$$\frac{\Gamma \vdash e_1 : t_1 \quad \Gamma \vdash e_2 : t_2 \quad t_1, t_2 \in \{\textbf{int}, \textbf{float}\}}{\Gamma \vdash e_1 / e_2 : \textbf{float}}$$

Division by zero remains a runtime condition because it depends on the evaluated denominator value.

6.4.4 Relational and Equality Operators

Relational operators require numeric operands and produce **bool**. Equality accepts equal primitive types or a numeric **int**/**float** pair.

$$\frac{\Gamma \vdash e_1 : t_1 \quad \Gamma \vdash e_2 : t_2 \quad t_1, t_2 \in \{\textbf{int}, \textbf{float}\}}{\Gamma \vdash e_1 < e_2 : \textbf{bool}}$$

$$\frac{\Gamma \vdash e_1 : t_1 \quad \Gamma \vdash e_2 : t_2 \quad (t_1 = t_2) \vee \text{numericPair}(t_1, t_2)}{\Gamma \vdash e_1 == e_2 : \textbf{bool}}$$

6.5 Statement Typing Rules

If statement. The condition must have type **bool**, and both reachable blocks must be well-typed:

$$\frac{\Gamma \vdash c : \textbf{bool} \qquad \Gamma \vdash B_1 \checkmark \qquad \Gamma \vdash B_2 \checkmark}{\Gamma \vdash \texttt{if}\ c\ B_1\ \texttt{else}\ B_2 \checkmark}$$

Routine call. The number of arguments must equal the number of parameters, and each argument type must be compatible with its corresponding parameter type.

Repeat statement. The repetition count is restricted by the grammar to an integer literal, so no separate expression inference is required for the count.

6.6 Two Passes of the Type Checker

1. Every routine signature is stored in a map. Conceptually, the structure is:

$$\texttt{routine\_name} \longrightarrow [(\texttt{param\_name}, \texttt{param\_type}), \ldots]$$

2. Routine bodies and top-level statements are checked. While a body is checked, its parameters are inserted into a temporary `type_env`.

The outer type environment is restored after each routine body, preventing one routine's parameters from leaking into another routine.

7 Names, Binding, Scope, and Lifetime

7.1 Static Scoping

Routines are declared only at global scope. When a routine is called, the parent of its new local environment is the global environment, not the caller's local environment.

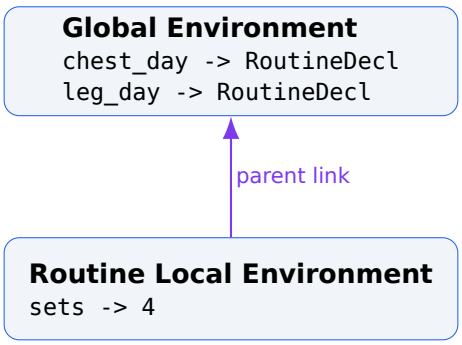


Figure 3: Static-scope environment chain during a routine call.

This structure guarantees lexical/static scoping. Under dynamic scoping, the parent would be the caller environment, making temporary names in the call chain visible to the callee.

7.2 Binding Times

Binding	Phase in Which It Occurs
Keyword → token category	During lexical analysis
Routine name → signature	Type-checker Pass 1
Routine name → RoutineDecl	Interpreter Pass 1
Parameter name → static type	While checking a routine body
Parameter name → runtime value	When the routine is called
Operator → behavior	During interpreter expression dispatch

7.3 Parameter Passing and Lifetime

Argument expressions are evaluated in the caller environment from left to right. The resulting values are copied into the new local environment, so the parameter-passing model is pass-by-value.

A local environment is created when a routine call begins and becomes unreachable when the call ends. Parameter lifetime is therefore stack-dynamic. Because PumpScript has no assignment, a parameter binding cannot be mutated inside the routine body.

8 Interpreter and Runtime Semantics

The interpreter executes the AST directly instead of translating it into another target language or bytecode. This architecture is commonly called a tree-walking interpreter. Statements are processed through execution handlers, while expressions are

recursively evaluated to values.

8.1 Statement Dispatch

The central `execute(node, env)` operation selects a handler according to the concrete statement node:

- `exec_exercise`
- `exec_rest`
- `exec_repeat`
- `exec_if`
- `exec_routine_call`
- sequential execution for each statement inside a `Block`

8.2 Expression Evaluation

A literal returns its stored Python value. An identifier is looked up through the environment chain. Unary and binary nodes recursively evaluate their operands. In a mixed numeric operation, an integer value is widened when a floating-point value is required.

> Binary operators evaluate the left operand before the right operand. Expressions currently have no side effects, so the final value would not change under a different order. Defining the order explicitly still makes the interpreter deterministic and prepares the language for possible future extensions.

8.3 Operational Semantics Summary

The notation $\langle S, \rho \rangle \Downarrow \rho'$ states that statement S executes under environment ρ and finishes in environment ρ'.

Rest statement.

$$\langle \texttt{rest}\ n\ \texttt{seconds}, \rho \rangle \Downarrow \rho \quad \text{and a formatted rest message is printed.}$$

Repeat statement.

$$\frac{n \geq 0 \qquad B \text{ is executed sequentially } n \text{ times}}{\langle \texttt{repeat}\ n\ \texttt{times}\ B, \rho \rangle \Downarrow \rho'}$$

The count is stored as a fixed integer. The interpreter still checks for a negative count defensively, even though ordinary source syntax cannot produce a negative `INT_LIT` for this grammar production.

If statement.

$$\frac{\langle c, \rho\rangle \Downarrow \text{true} \qquad \langle B_t, \rho\rangle \Downarrow \rho'}{\langle \texttt{if } c\ B_t \texttt{ else } B_f, \rho\rangle \Downarrow \rho'}$$

When the condition is false, the else block is executed if present; otherwise execution continues without modifying the environment.

Routine call.

1. Find the routine declaration in the global environment.
2. Evaluate arguments in the caller environment from left to right.
3. Create a new local environment whose parent is the global environment.
4. Bind parameter names to argument values using pass-by-value.
5. Execute the routine body in the local environment.

Exercise block. Every field expression is evaluated in the current environment. If the `weight` value is an integer, it is widened to floating point. The interpreter then validates positive `sets`/`reps` and non-negative `weight`, and prints a structured exercise record.

9 Error Handling

PumpScript separates errors by processing phase. Each phase has its own exception class and a consistent message format.

Phase	Exception	Examples Detected
Lexical	`LexerError`	Illegal character and unterminated string
Syntactic	`ParseError`	Missing parenthesis/brace, invalid statement start, missing type annotation
Static semantic	`TypeErrorPS`	Invalid field type, missing/duplicate field, non-Boolean condition, wrong argument, undefined identifier
Runtime	`RuntimeErrorPS`	Division by zero, non-positive sets/reps, negative weight, undefined routine

Table 4: Pipeline-oriented error classification.

9.1 Verified Error Examples

```
[Lexer Error] Line 4: Unexpected character '@'
```

```
[Parse Error] Line 10: Expected '}' but found ''
```

```
[Type Error] Line 6: In exercise 'squat': field 'weight' expects float but
got string
```

```
[Runtime Error] Line N: Division by zero
```

The interpreter is not called unless type checking succeeds. Therefore, errors that are statically decidable do not leak into runtime. Runtime checks are reserved for dynamic values such as the actual denominator or domain constraints involving evaluated numbers.

10 End-to-End Program Example

The following program contains a parameterized routine, two exercise blocks, a rest statement, and a top-level routine call.

Listing 1: A valid PumpScript program

```
// Valid Program: Chest Day Routine
routine chest_day(sets: int) {
    exercise bench_press {
        sets:      sets
        reps:      10
        weight:    80.0
        group:     "chest"
        completed: false
    }
    rest 60 seconds
    exercise chest_fly {
        sets:      sets
        reps:      12
        weight:    20.0
        group:     "chest"
        completed: false
    }
}

chest_day(4)
```

10.1 How the Program Moves Through the Pipeline

1. **Lexer:** emits keyword, identifier, literal, separator, and EOF tokens.
2. **Parser:** builds a `Program` AST containing one `RoutineDecl` and one `RoutineCall`.
3. **Type Checker Pass 1:** registers the signature `chest_day(int)`.
4. **Type Checker Pass 2:** resolves `sets` as **int**, checks every exercise field against the schema, and matches argument `4` with the integer parameter.
5. **Interpreter Pass 1:** stores the routine AST node in the global environment.

6. **Interpreter Pass 2:** creates a local environment for `chest_day(4)` and adds the binding `sets -> 4`.
7. Exercise field expressions are evaluated and formatted output is produced.

10.2 Observed Execution Output

```
==========================================
  Exercise: bench_press
==========================================
  sets        : 4
  reps        : 10
  weight      : 80.0 kg
  group       : chest
  completed   : no

  --- Rest: 60 seconds ---

==========================================
  Exercise: chest_fly
==========================================
  sets        : 4
  reps        : 12
  weight      : 20.0 kg
  group       : chest
  completed   : no
```

11 Technical Rationale for Major Design Decisions

A field such as `sets`, `weight`, or `completed` loses its domain meaning when it contains a value of the wrong type. Static typing detects such errors before execution. The trade-off is less flexibility and more explicit declarations than in a dynamically typed design.

Exercise blocks are not arbitrary dictionaries. They are domain records with fixed field names and types. This reduces structural flexibility but reliably rejects missing, duplicate, unknown, or semantically invalid fields.

Removing assignment and mutable local variables simplifies the environment and data-flow model. Routine parameters behave like read-only inputs. This reduces general-purpose computational power but makes the DSL easier to reason about and explain.

Routines describe workout actions rather than value-producing computations. Consequently, routine calls are statements and the language has no return statement. The execution model directly matches the domain model.

Because nested routines are not supported, closures are unnecessary. Making the global environment the parent of each routine-local environment provides static scoping with a compact and transparent implementation.

12 Testing Strategy and Validation

The implementation was validated with successful end-to-end programs and negative tests that target different layers of the pipeline.

Test	Target Layer	Expected Behavior
`valid1.pump`	End-to-end	Chest routine passes type checking and executes.
`valid2.pump`	End-to-end	Leg routine and multiple rest/exercise statements execute.
`valid3.pump`	Control flow	Multiple routines, `if/else`, and `repeat` execute.
`type_error.pump`	Type checker	A string value for `weight` is rejected.
`error1.pump`	Parser	A missing closing brace is reported.
`error2.pump`	Lexer	Illegal character `@` is reported.
`error3.pump`	Parser	A missing parameter type annotation is reported.
`error4.pump`	Lexer	An unterminated string is reported.
`error5.pump`	Parser	A routine call without parentheses is rejected.

12.1 Command-Line Usage

```
# Type-check and execute
python3 pumpscript.py examples/valid1.pump

# Print the AST without normal execution
python3 pumpscript.py examples/valid1.pump --dump-ast
```

The command-line entry point wraps each processing stage in a separate `try/except` boundary. When a stage fails, the pipeline stops immediately and no later stage is invoked.

13 Limitations and Future Extensions

The restricted scope of the current version is deliberate. Nevertheless, the language could be extended in the following directions:

- **Variable declarations and assignment:** storing computed values.
- **Arrays or lists:** managing collections of exercises programmatically.
- **Return values:** calculating total volume, duration, or other metrics.
- **Boolean connectives:** compound conditions using `and`, `or`, and `not`.
- **General record types:** user-defined structures and field access.
- **Bytecode or an intermediate representation:** improving performance beyond tree walking.
- **Source spans:** adding columns and character ranges to source locations.
- **Automated regression suite:** formal unit and integration testing.

Future changes should preserve the current type discipline, scope rules, and domain-specific simplicity. Otherwise, PumpScript could gradually lose its identity as a workout DSL and become a small general-purpose language.

14 Conclusion

PumpScript combines the main phases of language implementation in one working system:

Lexical Analysis → **Parsing** → **AST** → **Static Semantics** → **Interpretation**

Its main technical strengths are:

- a recursive-descent parser whose structure maps directly to the EBNF grammar,
- line-numbered and phase-specific error reporting,
- a strong type system that permits only safe numeric widening,
- a fixed exercise schema that enforces domain correctness,
- forward routine calls through two-pass registration,
- explicit static scoping through a global-parent environment model, and
- a dependency-free, modular Python implementation.

Although PumpScript is not a general-purpose language, it demonstrates how a lexer, parser, AST, type checker, and interpreter can be designed as a consistent whole. The central achievement is not merely the invention of new syntax; it is the integration of syntax, type rules, scope, runtime semantics, and error handling into one coherent executable language.

A Quick Language Reference

Construct	Syntax
Routine declaration	`routine name(param: type) { ... }`
Routine call	`name(argument)`
Exercise block	`exercise name { sets: ... reps: ... weight: ... group: ... completed: ... }`
Rest statement	`rest 60 seconds`
Repeat statement	`repeat 3 times { ... }`
Conditional	`if condition { ... } else { ... }`
Types	`int`, `float`, `string`, `bool`
Comments	`// line comment`

B Example Type-Error Program

```
routine leg_day(sets: int) {
    exercise squat {
        sets:      sets
        reps:      8
        weight:    "heavy"
        group:     "legs"
        completed: false
    }
}
leg_day(4)
```

```
[Type Error] Line 6: In exercise 'squat':
field 'weight' expects float but got string
```

This report is aligned with the PumpScript Part 2 source code and its verified example programs.